

Devoir 7 : Algorithmique et Chaînes de caractères
Calculatrice non autorisée

Exercice 1 : Extraction d'initiales (Analyse de script)

On considère la fonction Python suivante :

```
1 def initiales(identite):
2     init = identite[0]
3     for i in range(len(identite)):
4         if identite[i] == " " or identite[i] == "-":
5             init = init + identite[i+1]
6     return init.upper()
```

1. Décrire précisément ce que fait la ligne 2.
2. Expliquer le rôle de la boucle bornée associée à la condition `if` de la ligne 4.
3. À quoi sert l'instruction `.upper()` à la ligne 6 ?
4. Qu'affiche la fonction si l'utilisateur appelle les instructions suivantes :
 - a. `initiales("martin durand")`
 - b. `initiales("martin dupont durant")`
 - c. `initiales("martin dupont-durant")`
5. La fonction gère déjà le tiret (ligne 4). Que se passerait-il sans la condition `or identite[i] == "-"` pour le cas 4c ? Justifier.

Exercice 2 : Concaténation et itération (L'escalier)

On propose la fonction suivante :

```
1 def escalier(phrase):
2     affiche = phrase[0]
3     print(affiche)
4     for i in range(1, len(phrase)):
5         affiche = affiche + phrase[i]
6         print(affiche)
```

1. Que fait la ligne 2 ?
2. Expliquer le mécanisme de la boucle bornée (lignes 4 à 6).
3. Quel sera l'affichage complet si l'on appelle `escalier("Stylé")` ?
4. Si l'on remplace `range(1, len(phrase))` par `range(len(phrase))` à la ligne 4, que se passe-t-il à l'affichage ? Justifier.

Exercice 3 : Conversion de type (Fonction essai)

On donne la fonction suivante :

```
1 def essai(n):  
2     n = str(n)  
3     return len(n)
```

1. Quel est l'intérêt de l'instruction `str(n)` à la ligne 2 ?
2. Déterminer ce que renvoie la fonction pour les appels suivants :
 - a. `essai(123)`
 - b. `essai("Bonjour")`
 - c. `essai(-596)` — expliquer le résultat.
3. Modifier la fonction pour qu'elle renvoie le chiffre des unités d'un entier n quelconque en utilisant `str(n)` et l'indexation.

Exercice 4 : Approfondissement (Type et conversion)

On considère le programme suivant :

```
1 n = input("nombre : ")  
2 chiffres = str(n)  
3 print(chiffres[-2])
```

1. Qu'obtient-on si l'utilisateur saisit 1234 ?
2. Quel est le rôle de l'instruction `str(n)` ?
3. Transformer ce programme en une fonction `chiffre_dizaine(n)` qui renvoie le chiffre des dizaines d'un entier n .
4. Écrire une fonction `chiffre_unites(n)` qui renvoie le chiffre des unités.
5. Que se passe-t-il si l'utilisateur saisit 7 pour la fonction `chiffre_dizaine` ? Expliquer.

Exercice 5 : Cryptographie (Le Code César)

Le code César consiste à décaler chaque lettre d'un message d'un certain nombre de rangs dans l'alphabet. On choisit ici un décalage de **+3**.

1. Analyse manuelle :

a) En utilisant `ord()`, compléter le tableau suivant :

Lettre en clair	Code ASCII	ASCII + 3	Lettre codée
E			
C			
S			
A			
R			

b) Compléter la phrase : utiliser le code César revient à ajouter au code ASCII de chaque lettre, puis à appliquer `chr()` au résultat.

2. Étude de la fonction :

On donne la fonction `code_cesar` version naïve :

```

1 def code_cesar(message):
2     message = message.upper()
3     coder = ""
4     for caractere in message:
5         n = ord(caractere)
6         n_code = n + 3
7         lettre_c = chr(n_code)
8         coder = coder + lettre_c
9     return coder

```

a) Expliquer ce que fait la boucle bornée.

b) Déterminer le codage de **CESAR** et **ROYAUME**.

c) Quel(s) problème(s) apparaî(ssen)t avec les lettres **X**, **Y**, **Z** et avec un espace ? Déterminer le code ASCII de l'espace pour justifier.

3. Compléter la version corrigée ci-dessous pour résoudre ces problèmes :

```

1 def code_cesar(message):
2     message = message.upper()
3     coder = ""
4     for caractere in message:
5         n = ord(caractere)
6         if n == ..... :           # espace
7             n_code = .....
8         elif n + 3 > ..... :       # X, Y, Z
9             n_code = .....
10        else:
11            n_code = .....
12            lettre_c = chr(n_code)
13            coder = coder + lettre_c
14    return coder

```

d) Écrire le codage du mot **ZYGOTE** avec la version corrigée.